



FABRIC AUTOMATION AT SCALE

FROM CHAOS TO CONFIDENCE



WHO AM I?



Peer Grønnerup

Head of Engineering at Tabular Editor, Denmark

+15 years working with BI, Data Platform design & automation

Part of the Fabric Private Preview (Project Trident)

... and all in on Fabric & Power BI Automation and CI/CD!

 <https://www.linkedin.com/in/peergroennerup/>

 <https://peerinsights.emono.dev>

 <https://github.com/gronnerup>



ACCESS THE FABRICOPS REPOSITORY



<https://github.com/gronnerup/FabricOps>

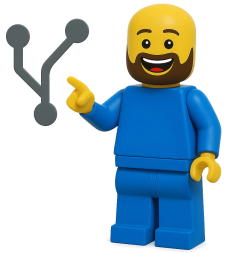


Disclaimer:

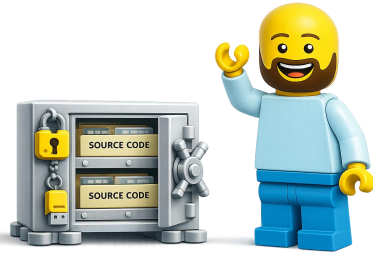
The solution demonstrated in this session is provided for **demonstration and educational purposes only**. It is **unsupported**, and there are no guarantees regarding functionality, stability, or future updates. You are free to use it **as-is** or modify it to fit your needs. Use at your own risk.



WHERE ARE WE GOING...



The basics of CI/CD
& Source Control



CI/CD & Source Control
in Microsoft Fabric



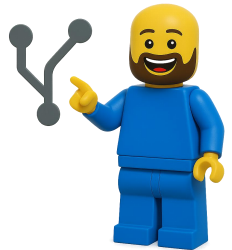
From Chaos...



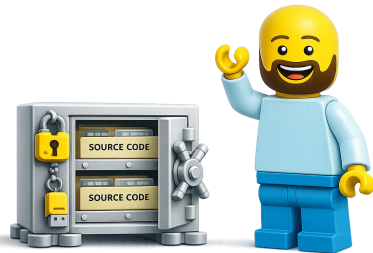
To Confidence...



WHERE ARE WE GOING...



The basics of CI/CD
& Source Control



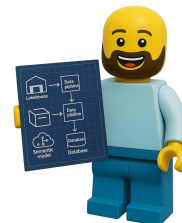
CI/CD & Source Control
in Microsoft Fabric



From Chaos...



To Confidence...



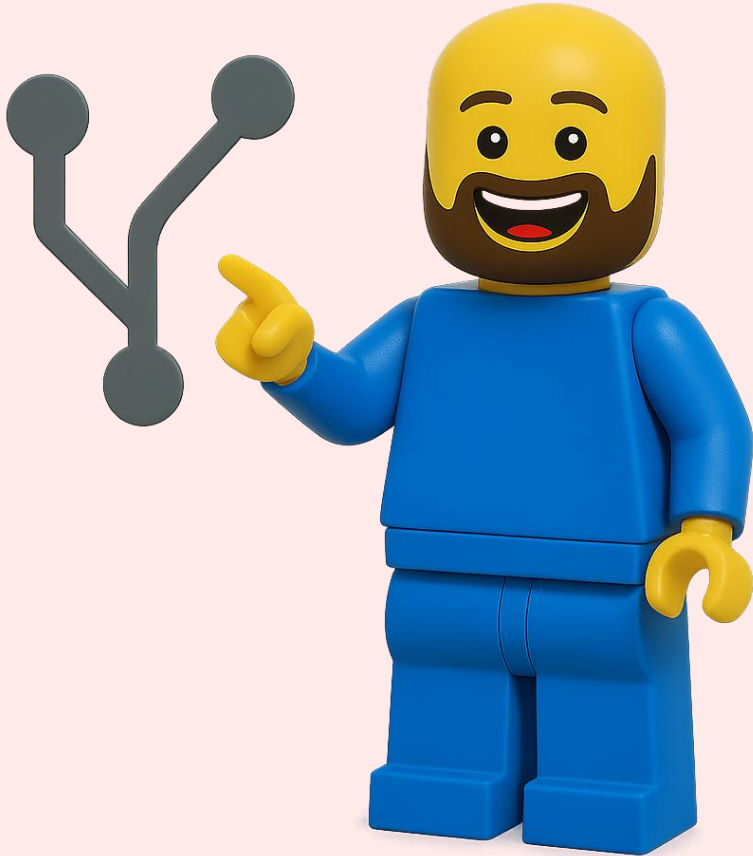
Architecting
for scale



Workspaces, repos
& environments



Automating CI/CD
and ways-of-working



THE BASICS OF CI/CD & SOURCE CONTROL



BASICS OF CI/CD & VERSION CONTROL

Git – Source Control (The Foundation)

- Your single source of truth - all code and definitions live here
- Versioning: tracks every change to Pipelines, Notebooks, Reports etc.
- Collaboration: feature branches prevent conflicts between developers
- Workspace sync: commit from Fabric to Git, update from Git to Fabric

CI/CD – The Automated Process

- Takes code from Git and delivers it safely to end users
- CI: automated tests on every commit to catch issues early
- CD: promotes changes through Dev → Test → Prod automatically
- Deployment Pipelines, ADO Pipelines and GitHub actions handle the flow



Git is your warehouse & history – CI/CD is the engine that safely delivers from factory to customer

MOTIVATION FOR CI/CD AND GIT?

CI – Continuous Integration

The Development Process

- Multiple developers work on the same codebase simultaneously
- Every commit triggers automated validation
- Catches integration issues early, before they reach production
- Enables code reviews via pull requests

CD – Continuous Deployment

The Release Process

- Automatically moves validated changes through stages
- Dev → Test → Production in a controlled flow
- Eliminates manual, error-prone deployments
- Consistent and repeatable releases every time

Git – Version Control

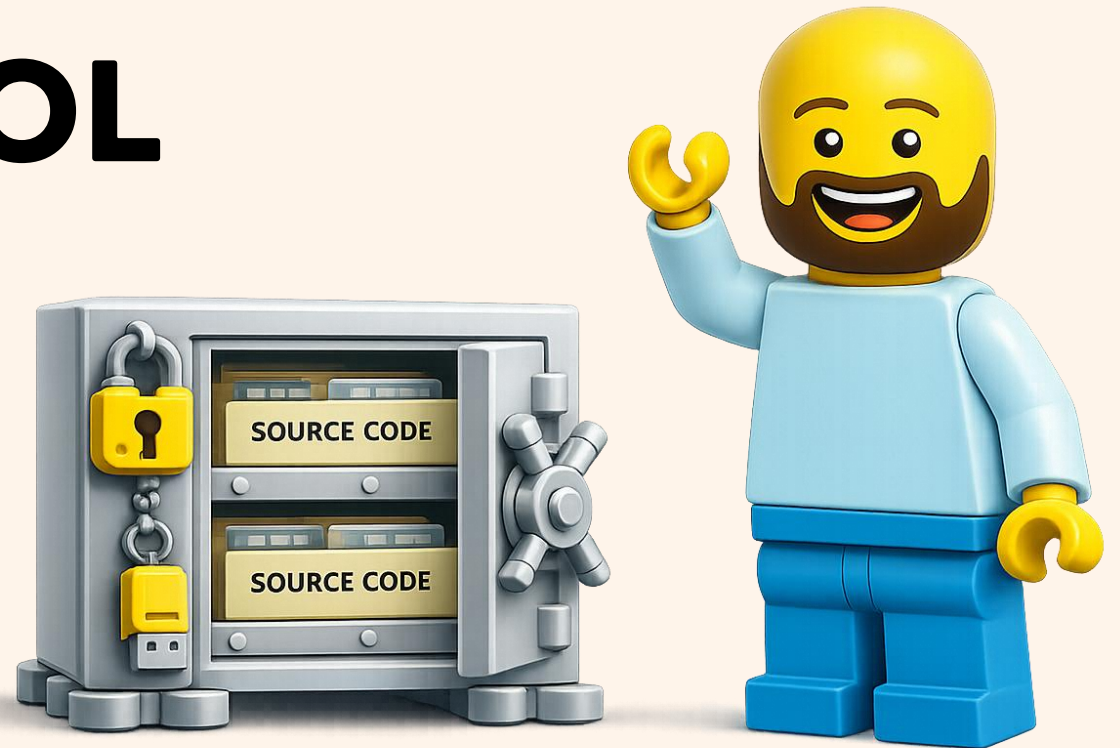
The Foundation

- Tracks every change with full history and rollback
- Feature branches isolate work-in-progress safely
- Supports experimentation without risk to production
- Single source of truth for all code and definitions

Together they form the backbone of Application Lifecycle Management (ALM) in Microsoft Fabric



CI/CD AND SOURCE CONTROL IN FABRIC



THE FABRIC CI/CD PLATFORM

Git Integration

Deployment Pipelines

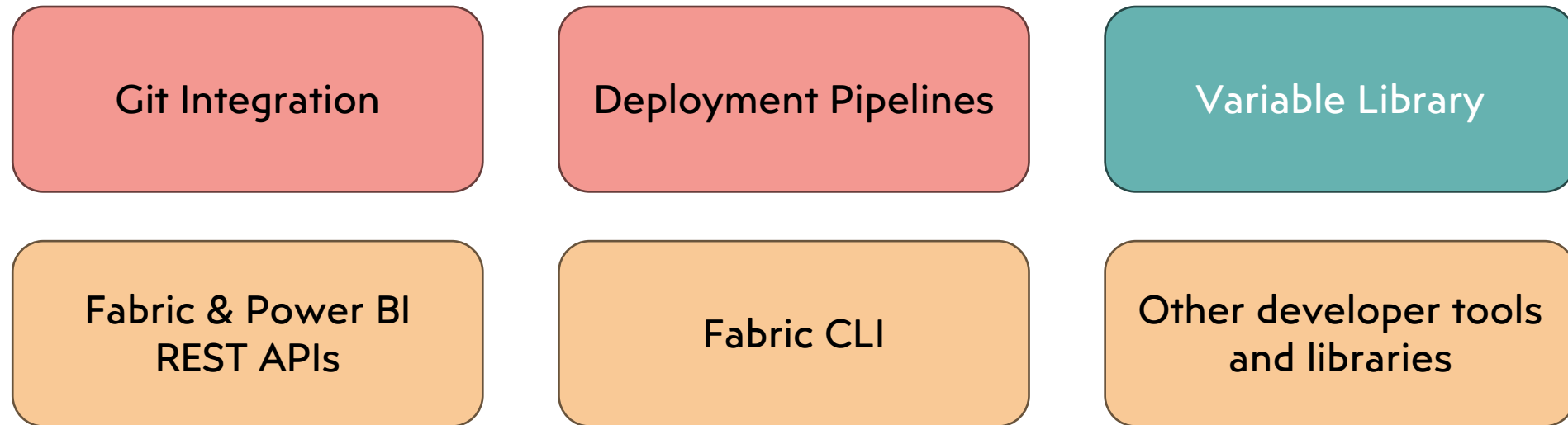
Variable Library

Fabric & Power BI
REST APIs

Fabric CLI

Other developer tools
and libraries

THE FABRIC CI/CD PLATFORM



FABRIC GIT INTEGRATION



How It Works

Git Providers: Azure DevOps | GitHub
Including cross-tenant ADO support and Service Principal for GitHub

Configured at Workspace Level
Connect workspace → repo → branch → folder

- Commit changes from workspace to Git
- Update workspace from Git (sync)
- Branch out to isolated feature workspaces
- Workspace folder structure preserved in repo
- PR-based workflow with code reviews

Authentication options

 Github	 DevOps
Github PAT	OAuth 2.0 Service Principal

Read more: <https://learn.microsoft.com/en-us/fabric/cicd/git-integration/intro-to-git-integration>



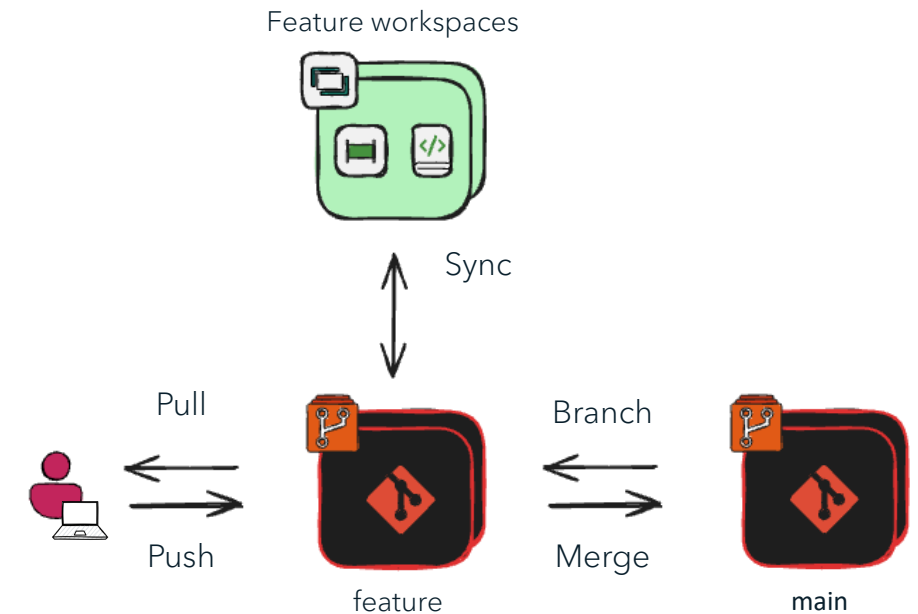
DEMO TIME!



CONTINUOUS INTEGRATION

Development flow

1. Create new feature branch from main
2. Create isolated development workspaces
3. Implement changes
4. Commit changes back to feature branch
5. Create PR and merge changes to main
6. Delete feature branch + isolate development workspaces



<https://learn.microsoft.com/en-us/fabric/cicd/best-practices-cicd>

FABRIC GIT INTEGRATION



How It Works

Git Providers: Azure DevOps | GitHub
Including cross-tenant ADO support and Service Principal for GitHub

Configured at Workspace Level
Connect workspace → repo → branch → folder

- Commit changes from workspace to Git
- Update workspace from Git (sync)
- Branch out to isolated feature workspaces
- Workspace folder structure preserved in repo
- PR-based workflow with code reviews



New Capabilities *FabCon 2026*

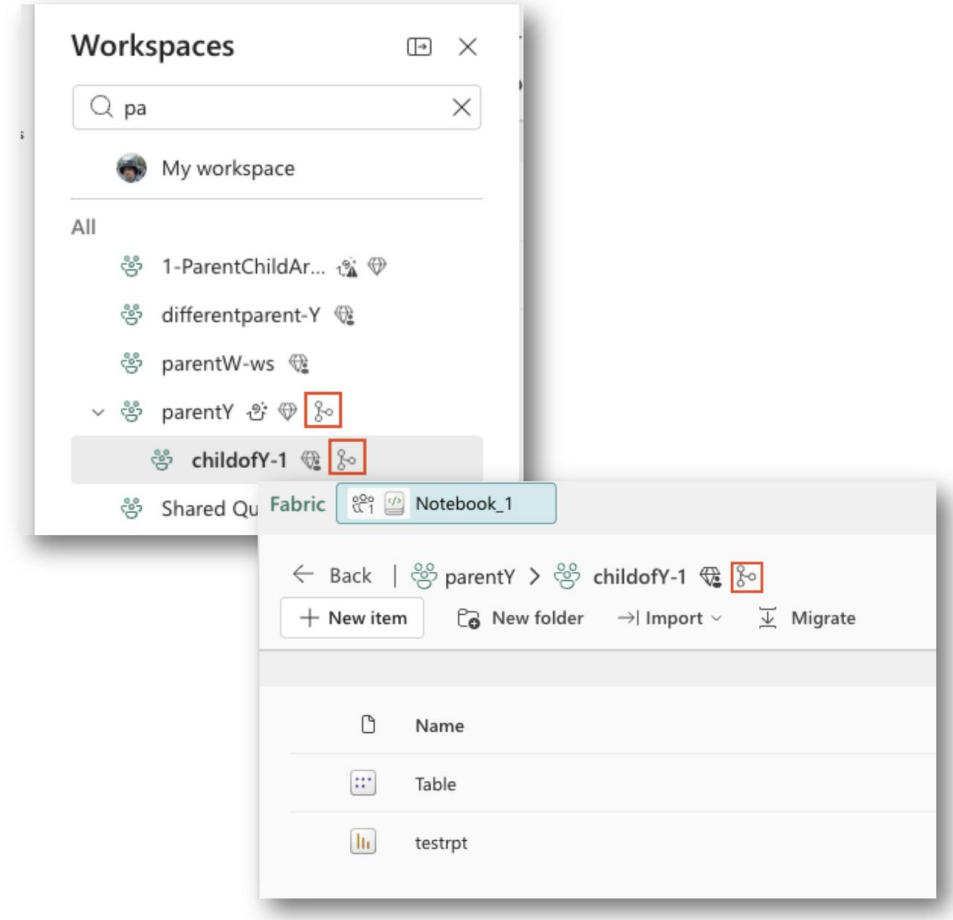
- **Selective Branching** – Branch out with only the items you need
- **Compare Code Changes** – Rich diff view – see exactly what changed before sync
- **Branched Workspaces** – Visual parent-child relationship between workspaces
- **Variable Library** – Manage configs across Dev/Test/Prod – no hardcoding
- **Fabric MCP + VS Code** – GitHub Copilot talks to Fabric natively (GA)
- **Extensibility Toolkit** – Full CI/CD for custom workloads (GA)

Read more: <https://learn.microsoft.com/en-us/fabric/cicd/git-integration/intro-to-git-integration>

BRANCHED WORKSPACE

PREVIEW

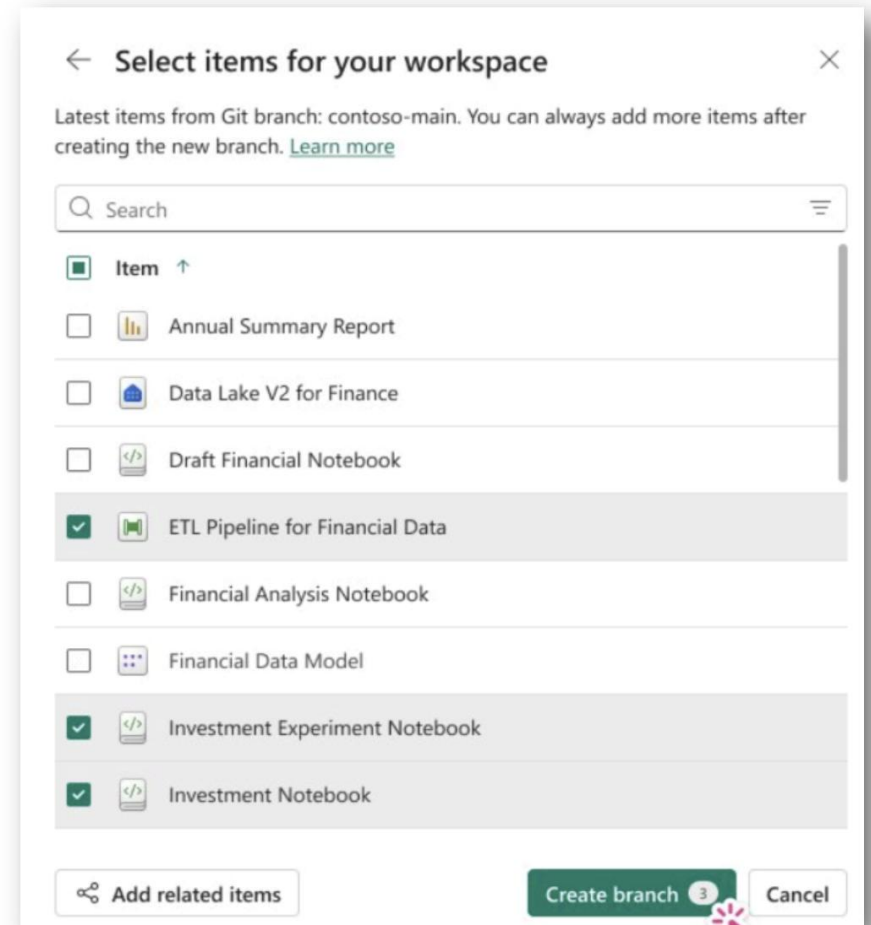
- Creates a relation during the branch-out operation
 - Branch into a new workspace or an existing one
- The branched workspace acts as a logical “child” of the source workspace
- New visual indicators for Git-connected workspaces
- Quick navigation into connected Branch workspaces
- Foundation for future enhancements



SELECTIVE BRANCHING

PREVIEW

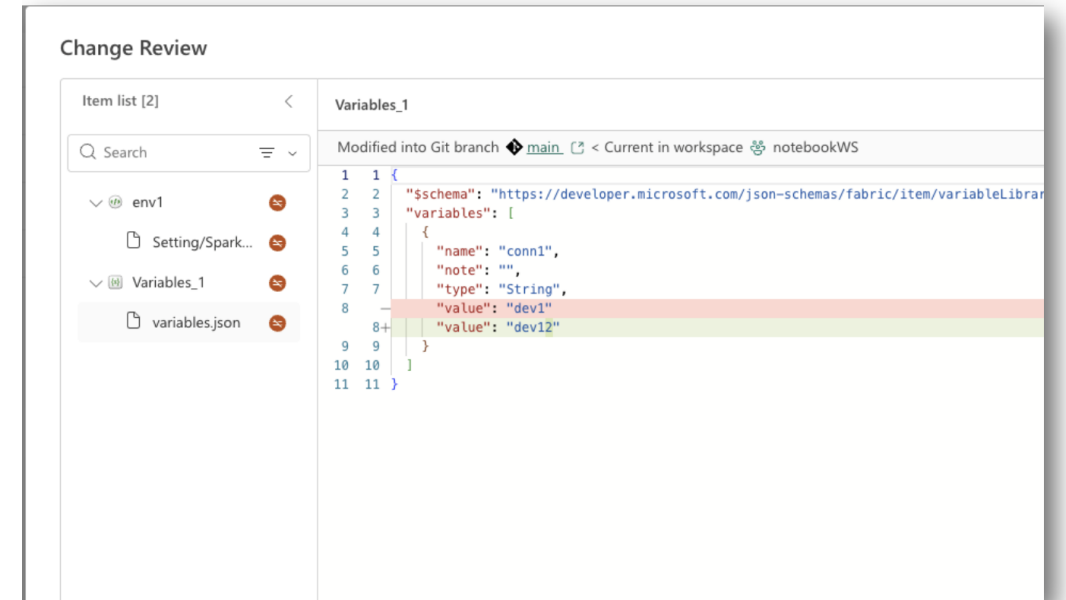
- New 'selective' mode when branching-out
- Choose subset of the items instead of the entire workspace items
- Faster way to start working with branch workspace
- Import only on what you need
- Add new items iteratively



COMPARE CHANGES

PREVIEW

- Review workspace changes before committing them to Git
- Inspect incoming updates before performing an "Update All"
- Identify and resolve conflicts with full visibility before applying changes





CONTINUOUS DEPLOYMENT

Fabric Deployment pipelines



- No code experience
- For simpler solutions
- No support for *:
 - Pre-deployment opr.
 - Post-deployment opr.
 - Test & validation

Git based deployment



- Suitable when using Gitflow
- Each environment connected to a dedicated git branch
- No need for building environments
- Might require post-deployment operations

Git based deployment using build environment



- Build environments
- Deployment through ADO Pipelines & Github actions
- Supports manipulation of item definitions

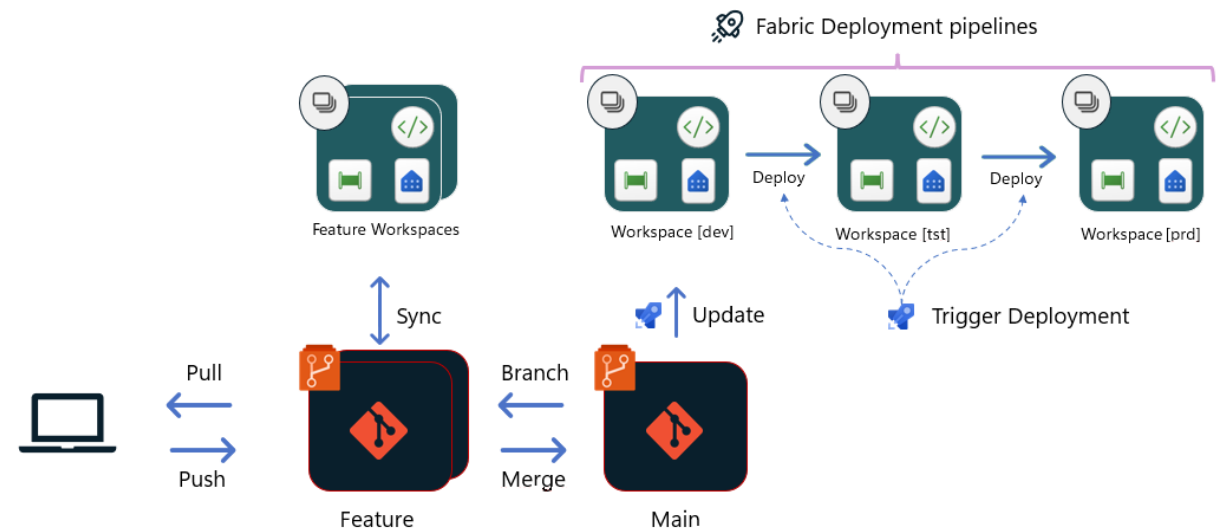


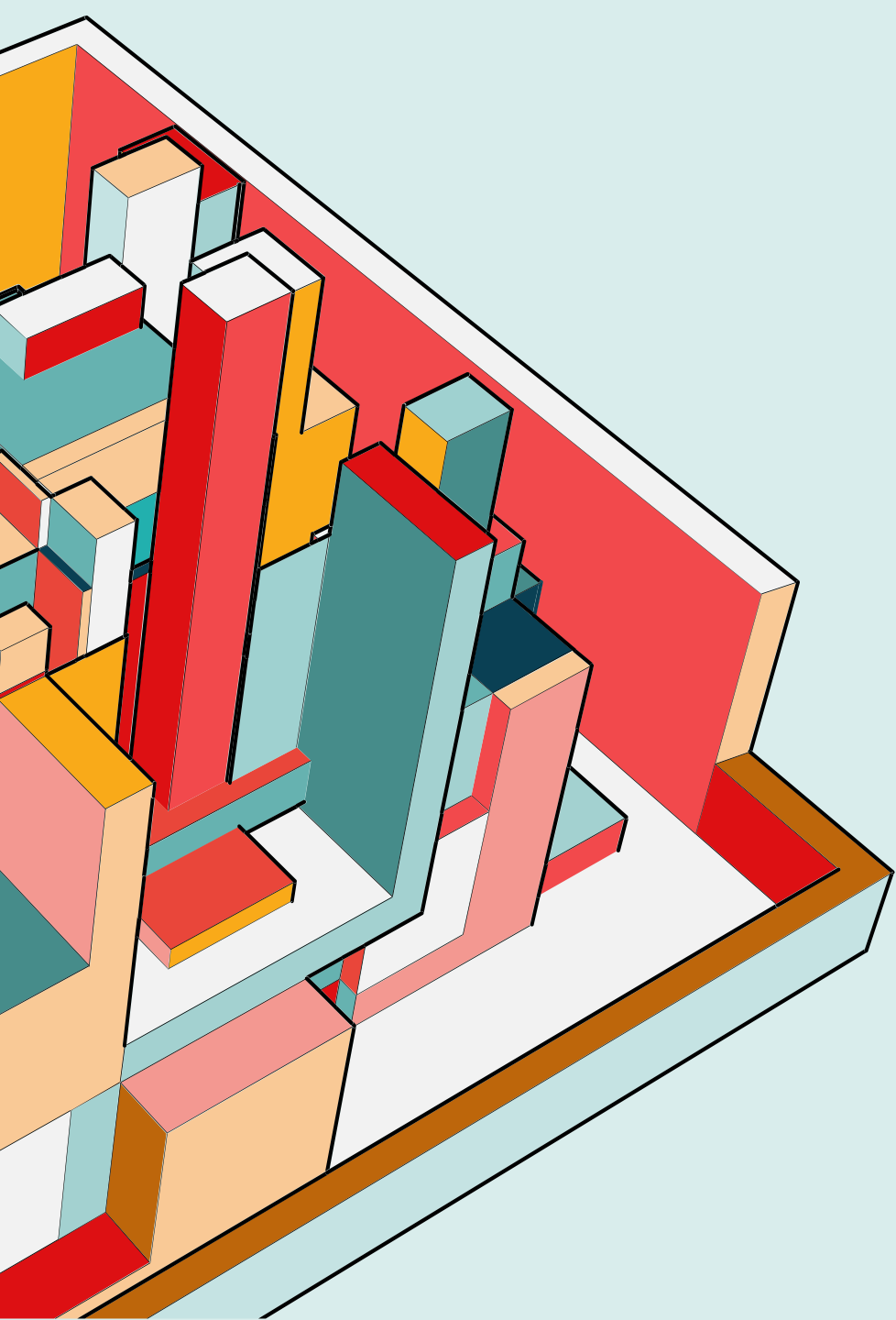
CONTINUOUS DEPLOYMENT

Fabric Deployment pipelines



- Use GIT integration for CI
- User Deployment Pipeline for CD
- **main** branch is integration branch
- dev workspace kept in sync with **main**
- Release using pipeline Deploy action

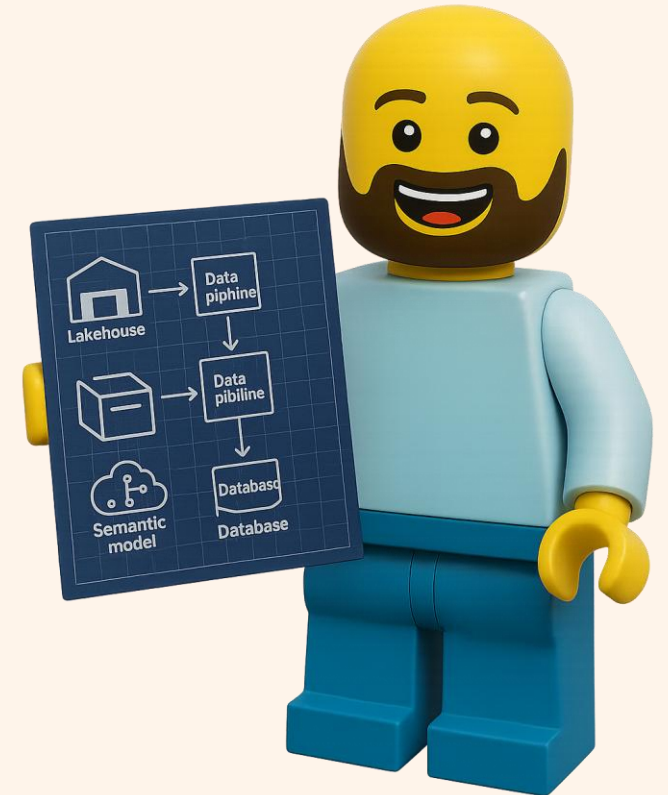




FROM CHAOS TO CONFIDENCE

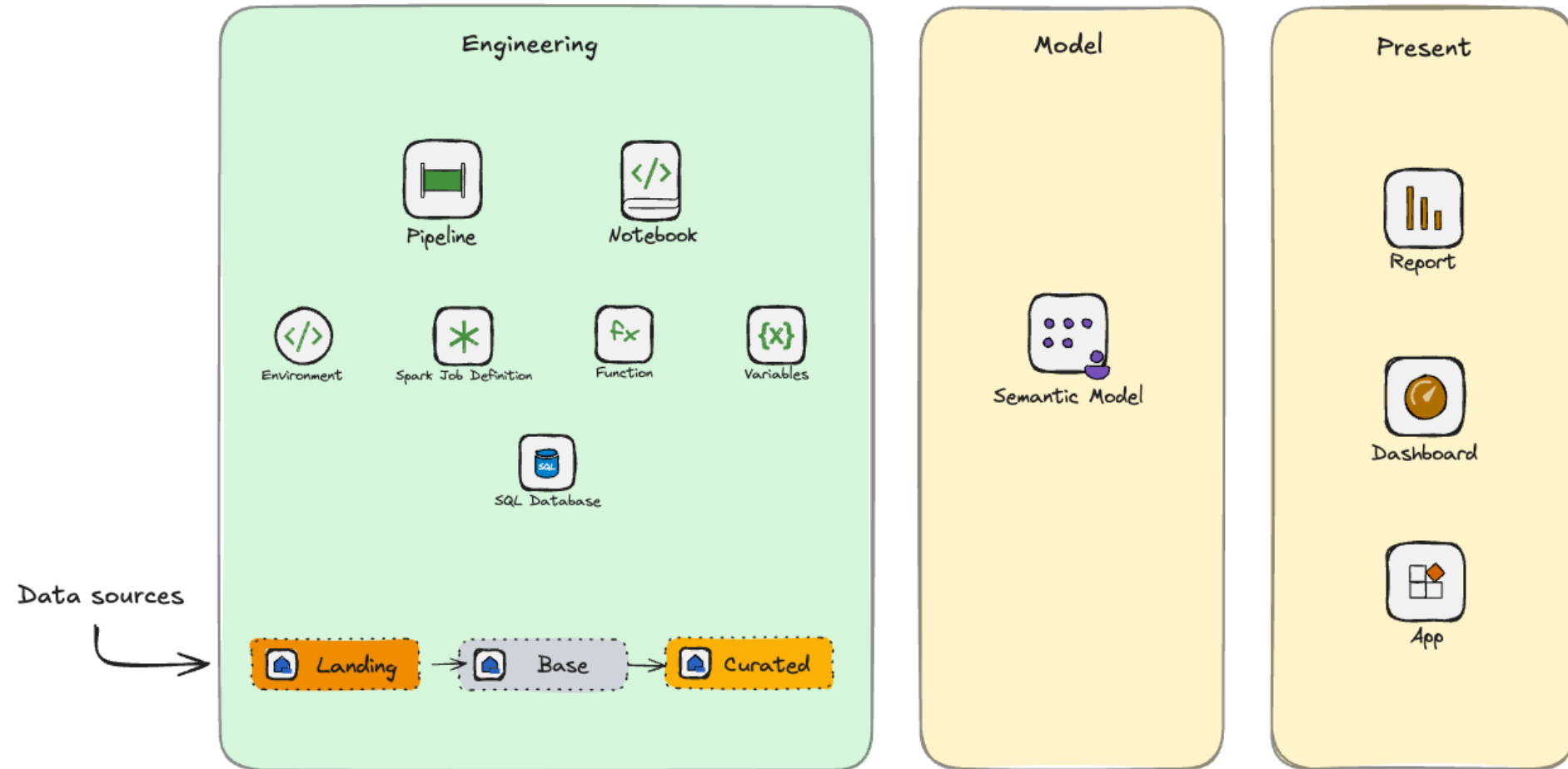
ENTERPRISE CI/CD AT SCALE

ARCHITECTING FOR SCALE



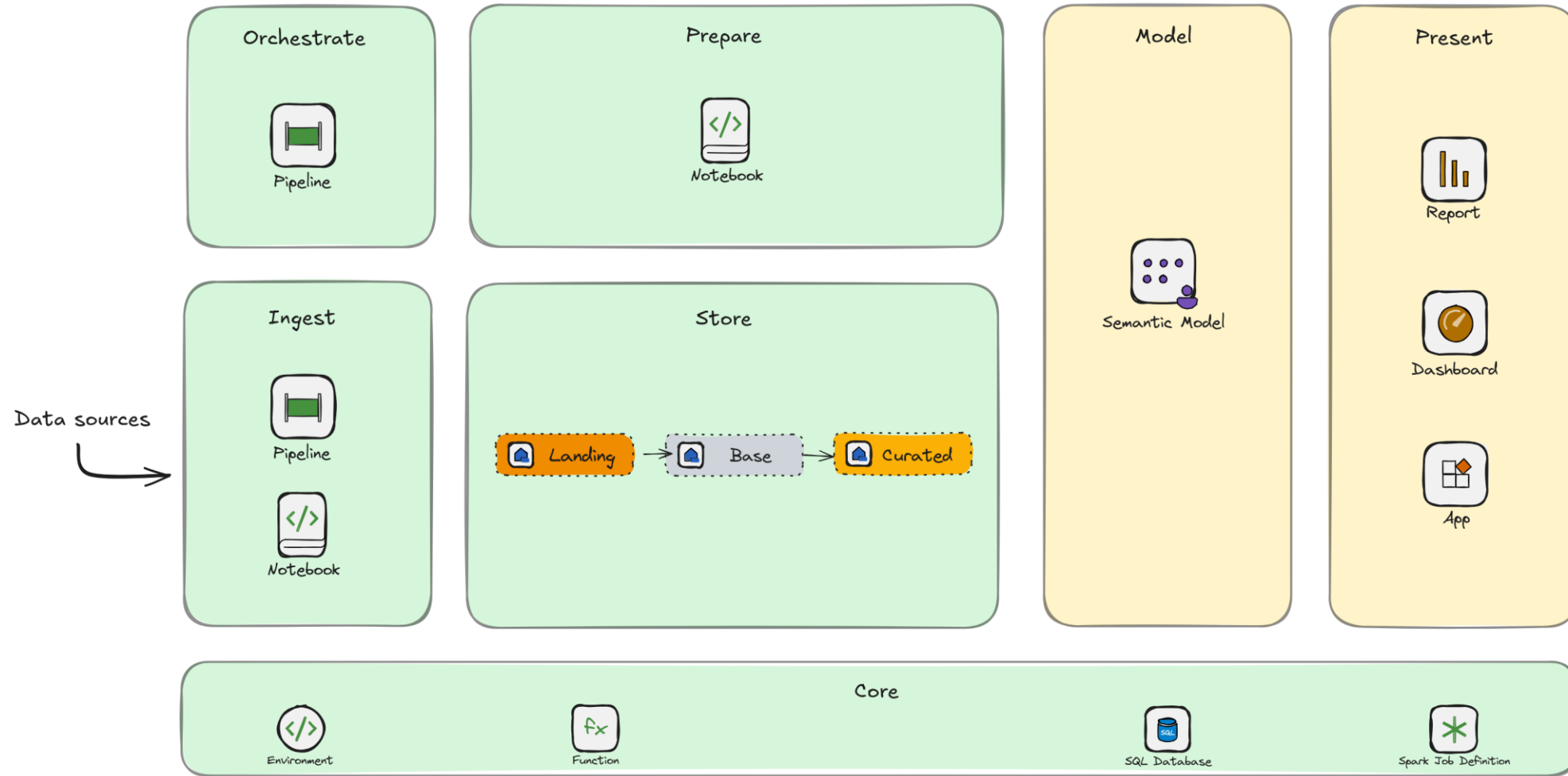


REFERENCE ARCHITECTURE - BASIC



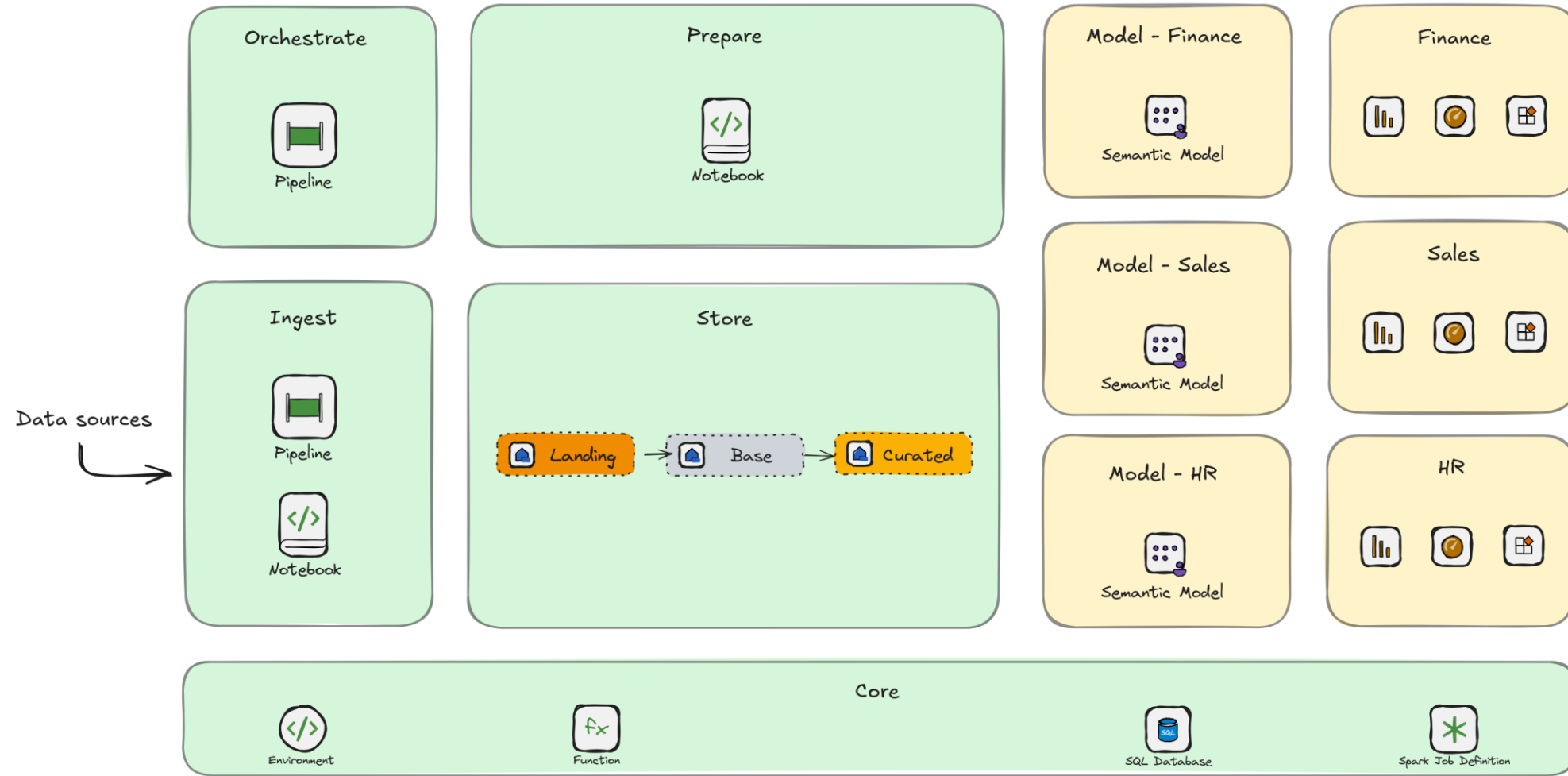


REFERENCE ARCHITECTURE - ENTERPRISE





REFERENCE ARCHITECTURE - ENTERPRISE+

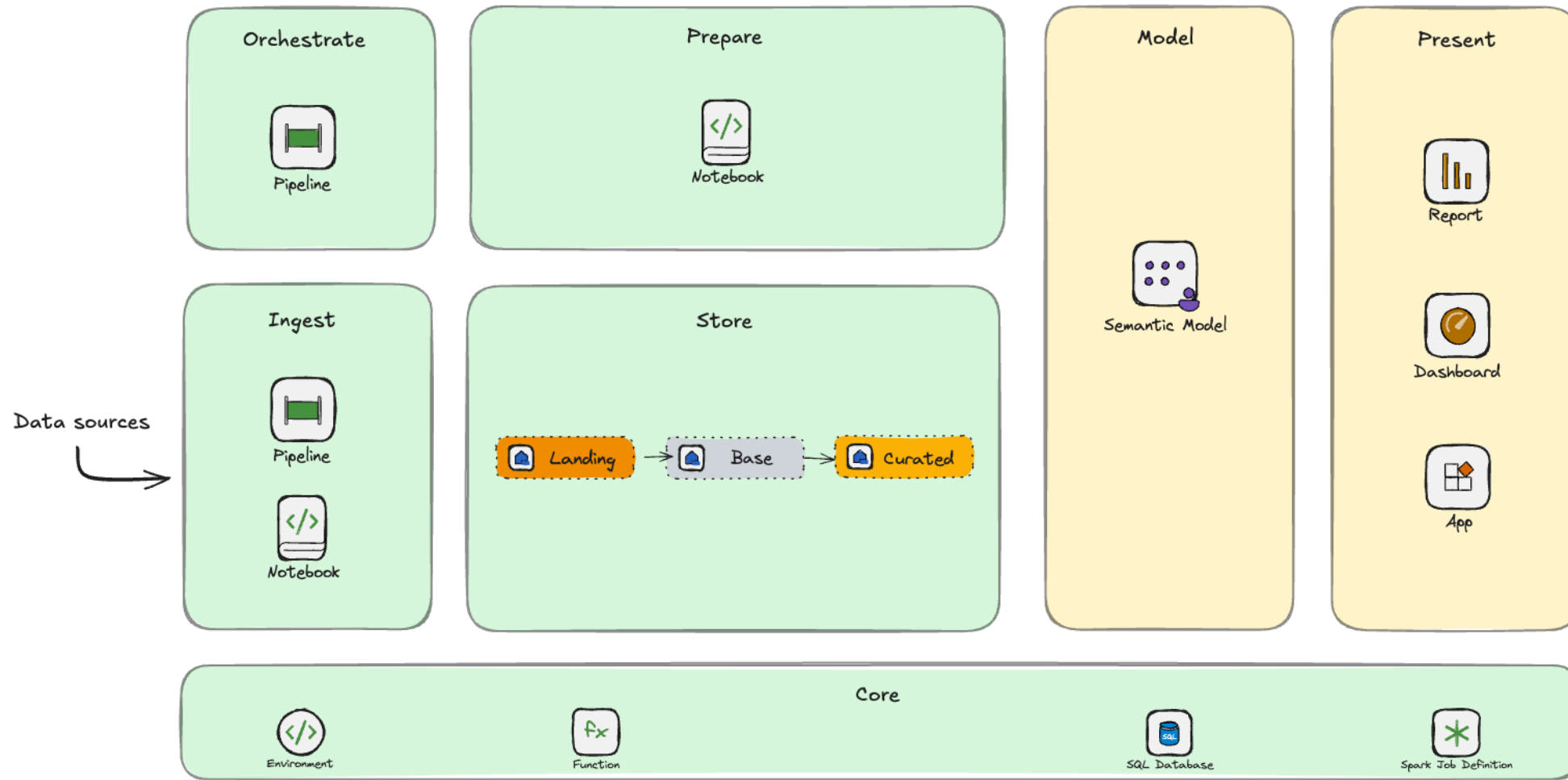


WORKSPACES, REPOS & ENVIRONMENTS



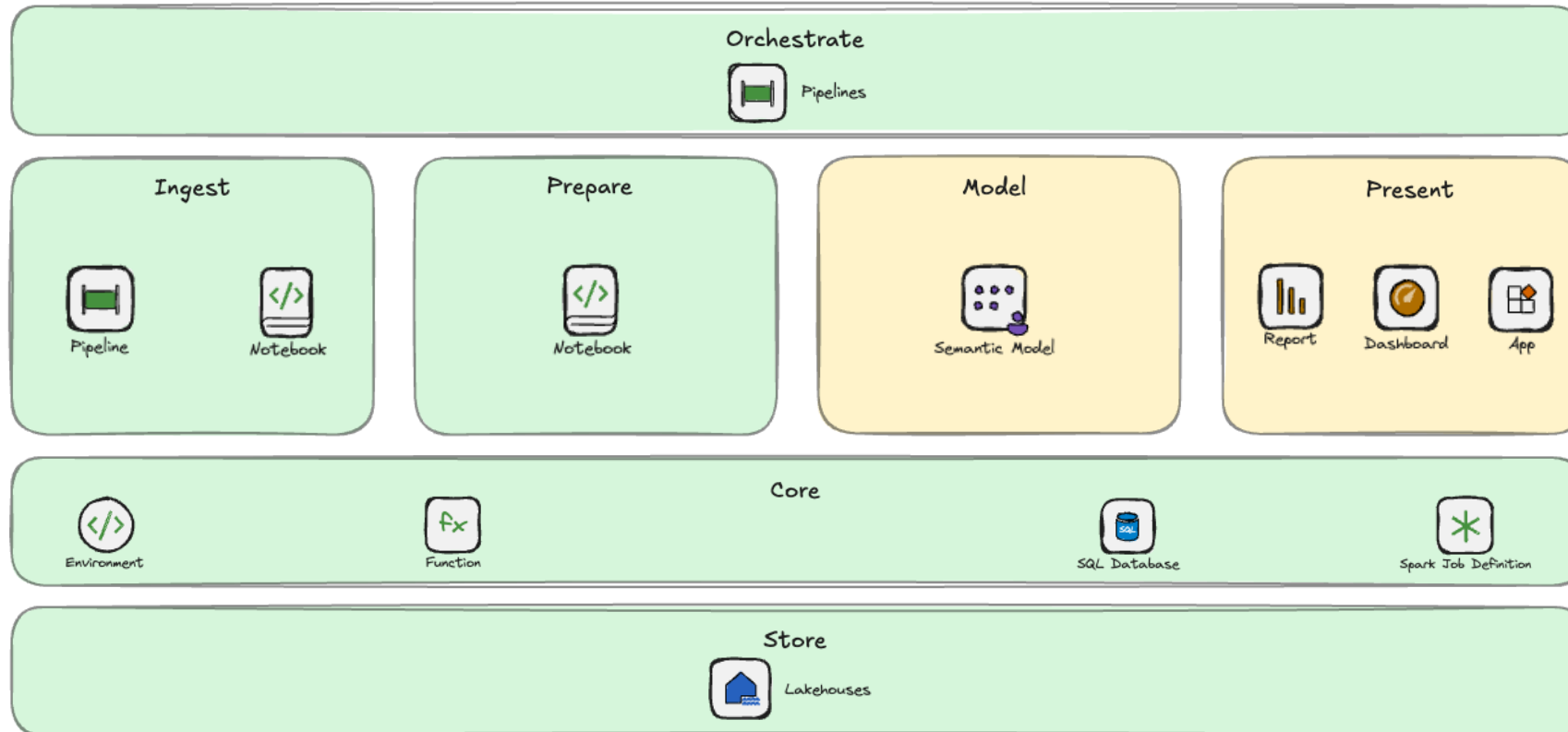


REFERENCE ARCHITECTURE - ENTERPRISE





FABRIC WORKSPACE STRUCTURE

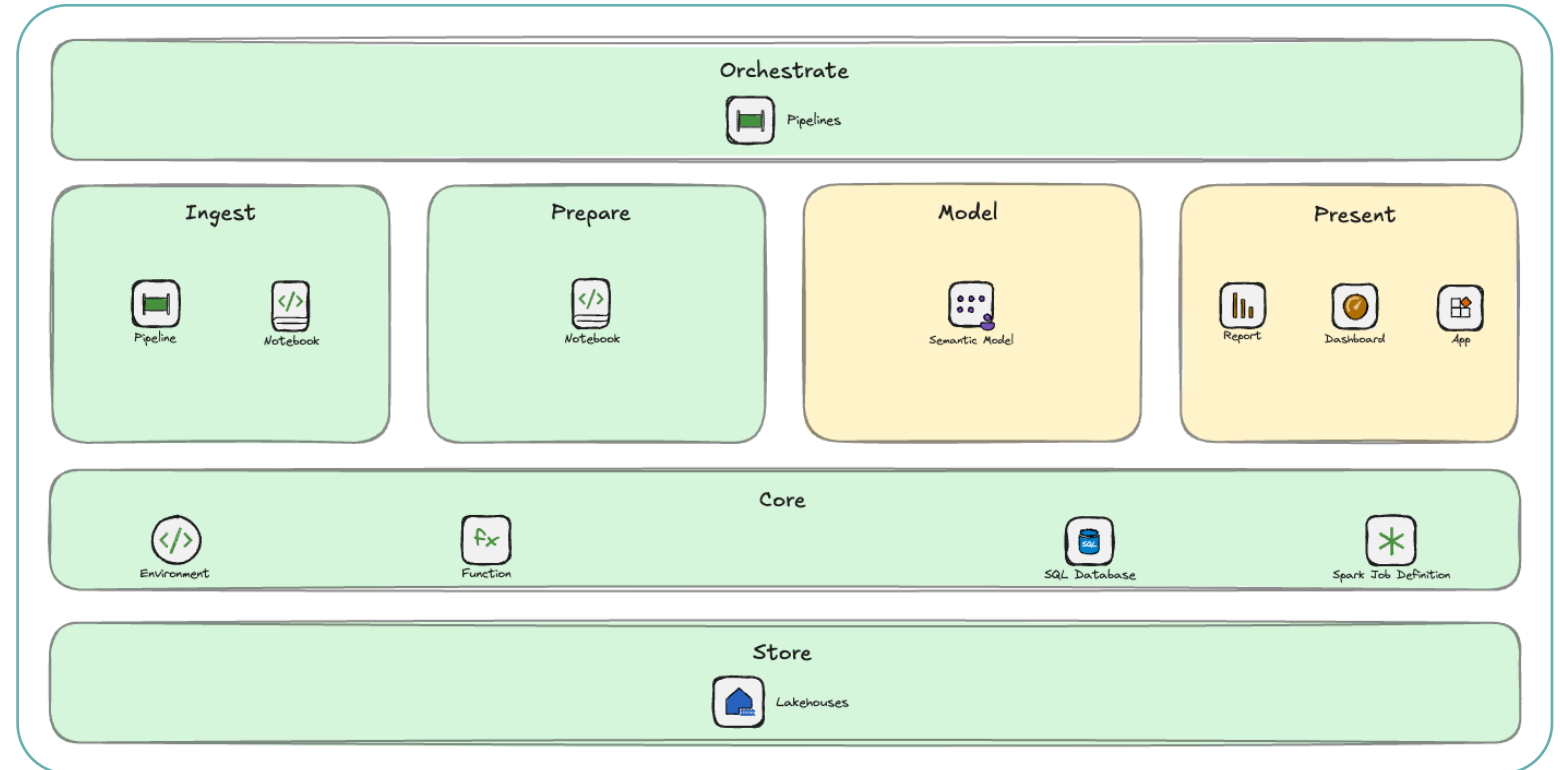


x3
[dev, tst, prd]



FABRIC WORKSPACE STRUCTURE

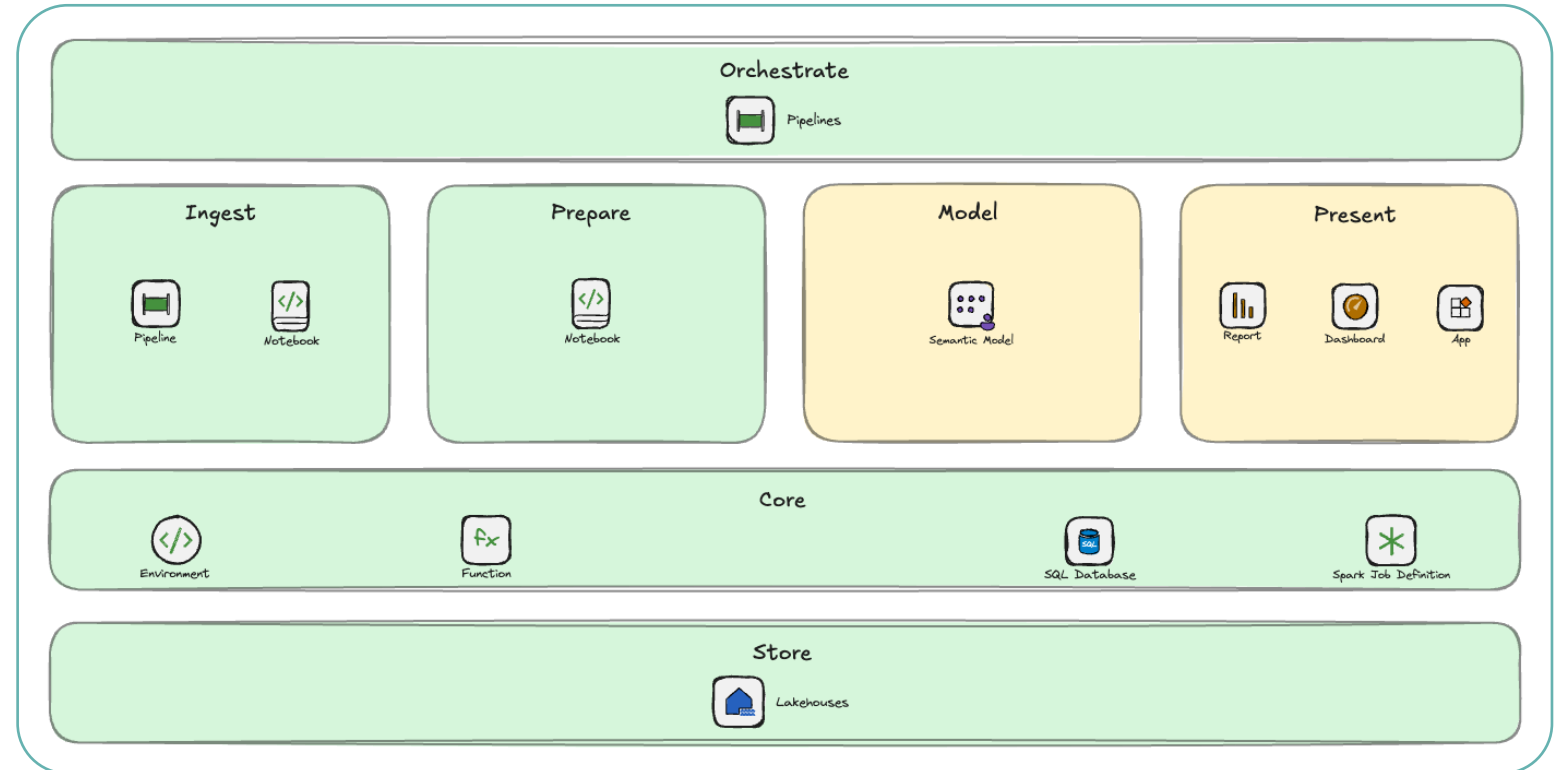
- Security and Access
- Separation of duties
- Network & connectivity
- Capacity isolation
- Governance & compliance
- Testing & Deployment
- Item organization
- Git integration and CI/CD





FABRIC WORKSPACE STRUCTURE

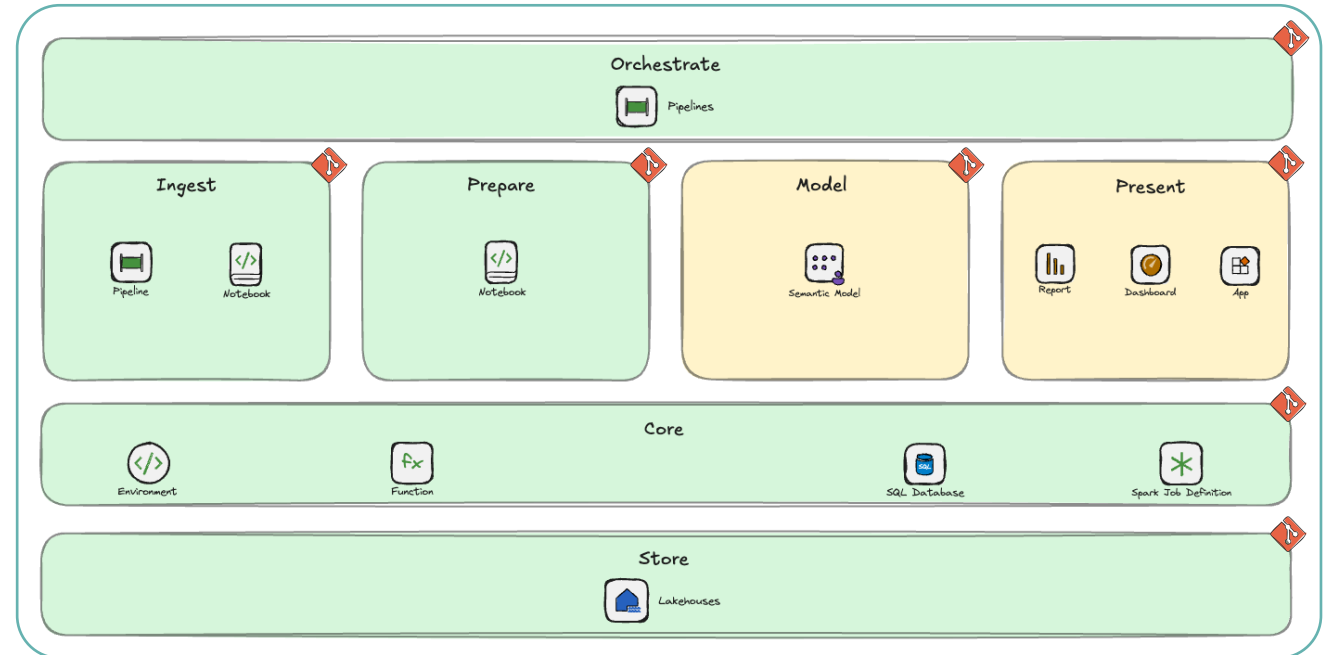
- Security and Access
- Separation of duties
- Network & connectivity
- Capacity isolation
- Governance & compliance
- Testing & Deployment
- Item organization
- **Git integration and CI/CD**





MY GO-TO FOR ENTERPRISE GIT SETUP

- Organize your workspaces by layer
- Use a clear naming conventions
- Use the Git provider of your choice
- Branching strategy depends...
- Setup Git integration on selected workspaces in development
- Use a mono repo approach





MY GO-TO FOR ENTERPRISE GIT SETUP



Git Repo structure

.azure-pipelines

.github

automation

documentation

solution

/core

/ingest

/model

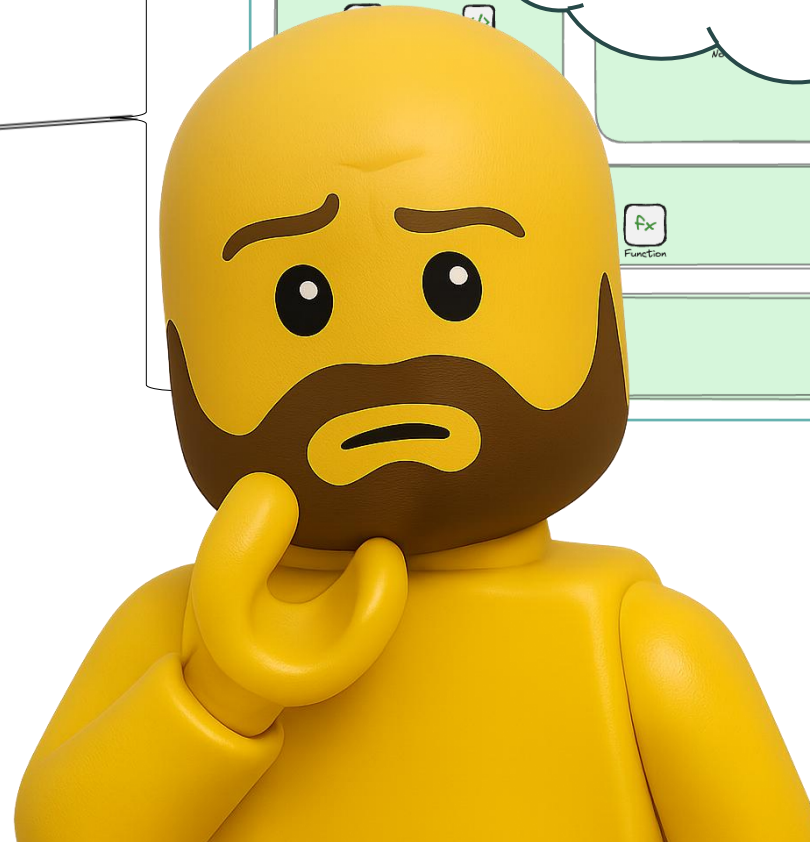
/orchestrate

/prepare

/present

/store

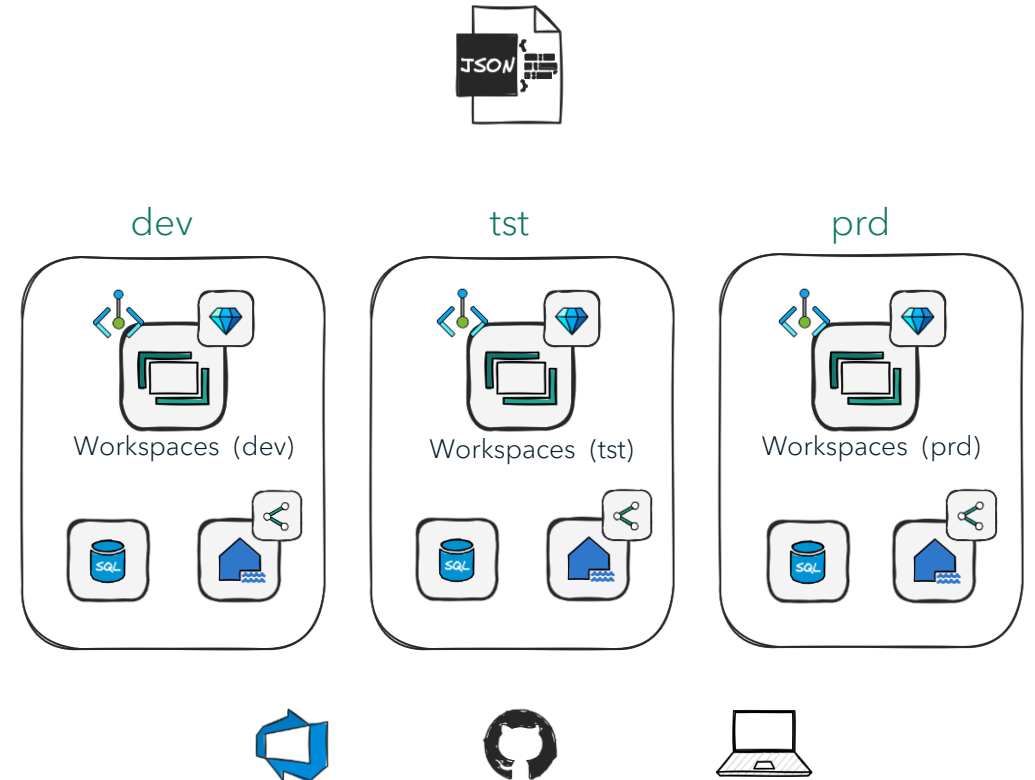
But wait! How
do I get all this
done?





AUTOMATING SOLUTION SETUP

- Recipe based solution (json)
- Utilizes Python and the Fabric CLI
- Can run from Azure DevOps, GitHub or from any client machine running python
- Minimum requirements:
 - A Service Principal
 - A Fabric Capacity



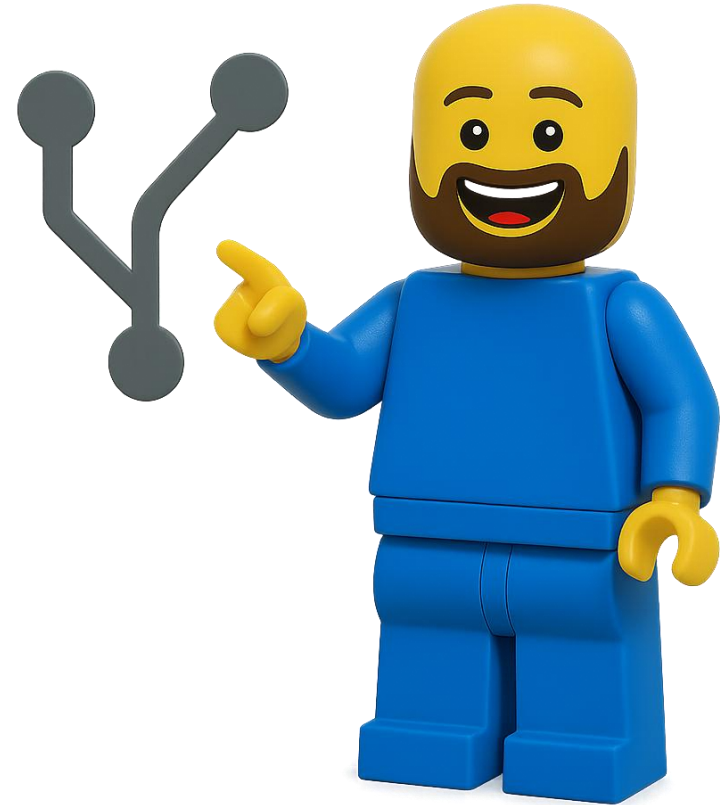
Download source code: <https://github.com/gronnerup/FabricOps>



SHOW TIME!

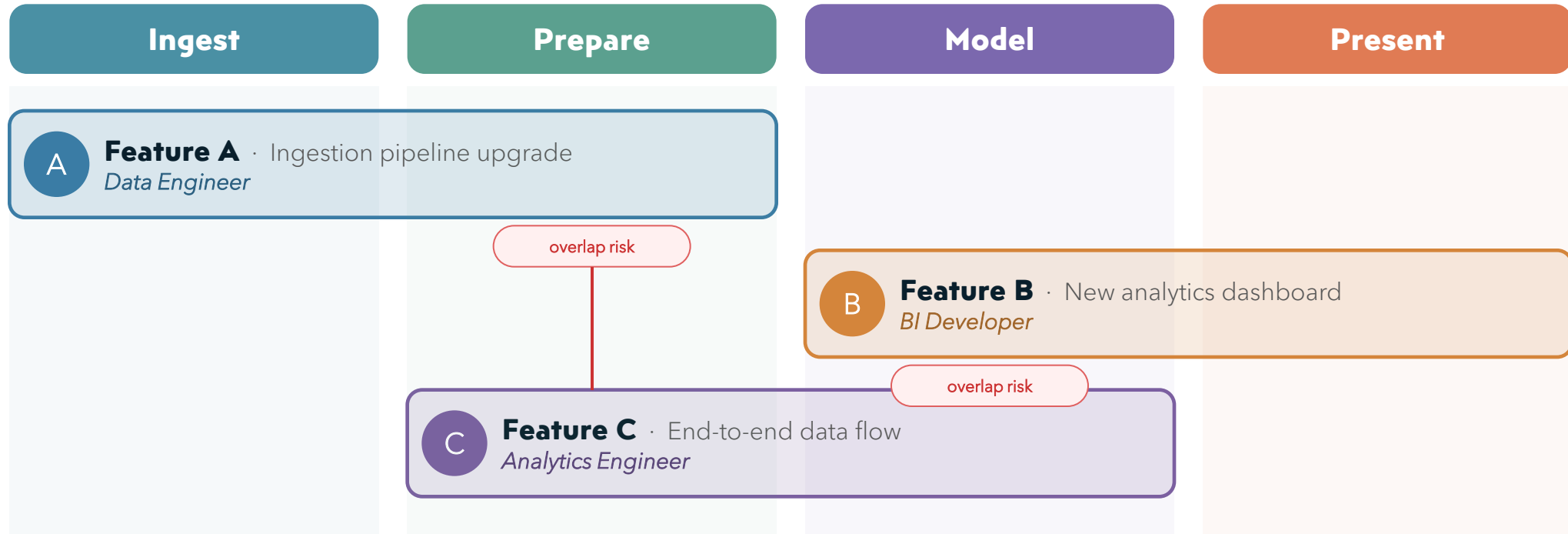


SUPPORTING REAL-WORLD COLLABORATION





HOW TO SUPPORT MULTIPLE DEVELOPERS?



Real-world collaboration needs isolation **Branching is the answer**

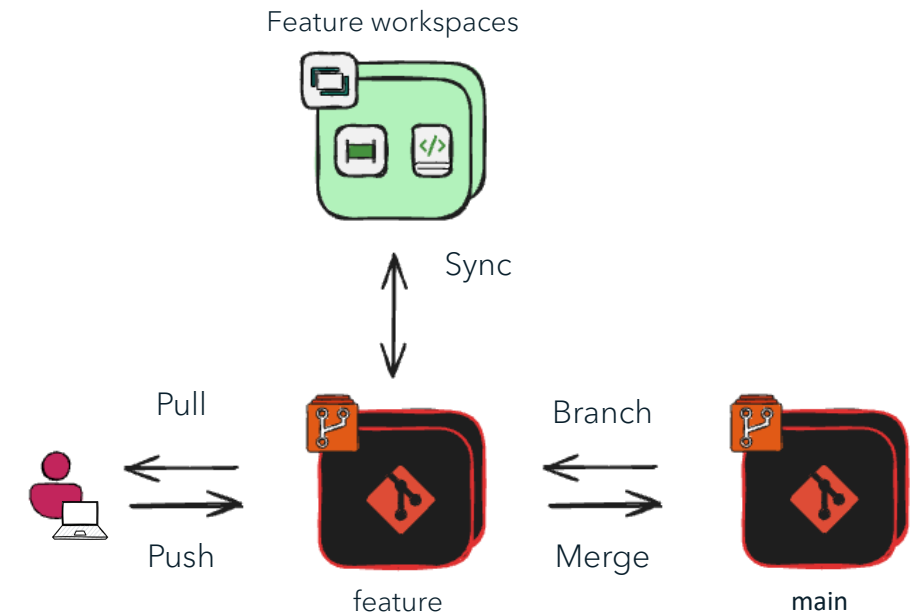
Fabric Git Integration: Workspace linked to specific repository, branch and Git folder



GENERAL DEVELOPMENT WORKFLOW

Development flow - Supported by automation!

1. Create new feature branch from main
2. Create isolated development workspaces
3. Implement changes
4. Commit changes back to feature branch
5. Create PR and merge changes to main
6. Delete feature branch + isolate development workspaces



<https://learn.microsoft.com/en-us/fabric/cicd/best-practices-cicd>



WHY AUTOMATE THIS FLOW?

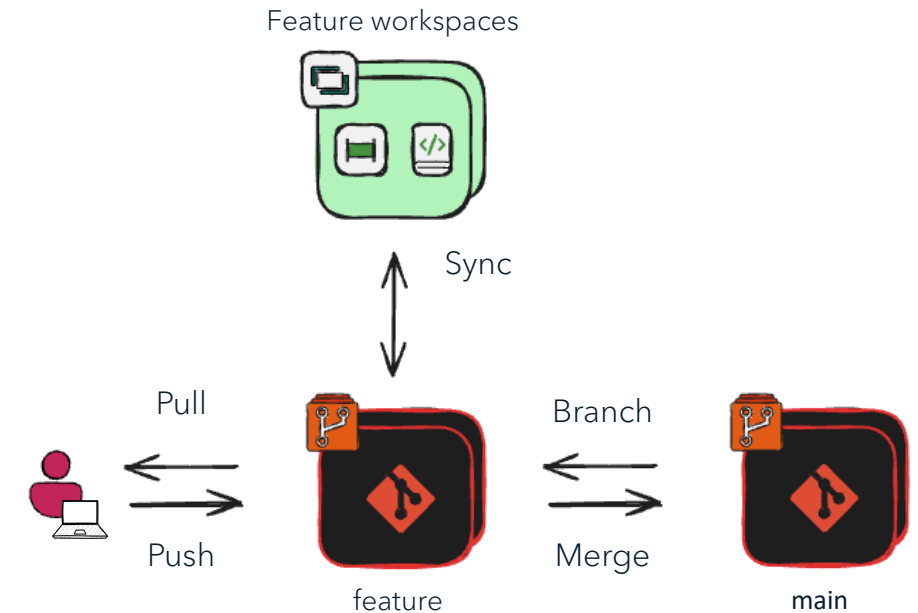
🖱️ ClickOps

- ✗ Manual process
- ✗ 1 workspace – 1 branch
- ✗ No transfer/setup of ACL, Spark settings, Private Endpoints, WS Identity
- ✗ Inherits source capacity
- ✗ Requires manual cleanup

VS

⚡ Automated

- ✓ Fully automated – just create the branch
- ✓ Multiple workspaces supported per branch
- ✓ Customize ACL, Spark settings, Private Endpoints, WS Identity, Capacity
- ✓ Automated sync & cleanup



<https://peerinsights.emono.dk/automating-feature-workspace-maintenance-in-microsoft-fabric>

<https://justb.dk/blog/2025/02/fabric-spark-notebooks-and-cu-consumption/>



DEMO TIME!



AUTOMATING DEPLOYMENT





RELEASE PROCESS – THE MAIN OPTIONS...

Fabric Deployment pipelines



- No code experience
- For simpler solutions
- No support for *:
 - Pre-deployment opr.
 - Post-deployment opr.
 - Test & validation

Git based deployment



- Suitable when using Gitflow
- Each environment connected to a dedicated git branch
- No need for building environments
- Might require post-deployment operations

Git based deployment using build environment



- Build environments
- Deployment through ADO Pipelines & Github actions
- Supports manipulation of item definitions



ENTERPRISE CI/CD BUILDING BLOCKS

Tools & automation options



Azure DevOps Pipelines



Github actions



Python scripts & wrappers



Fabric CLI wrappers



Tabular Editor 2 CLI (free)

fabric-cicd Python Library

**NOW OFFICIALLY
SUPPORTED**



Developer friendly

- Code-first approach
- No need to call Fabric APIs directly



Environment supports

- Parameterization
- Reusable for locale, DevOps and more...



Smart deployment

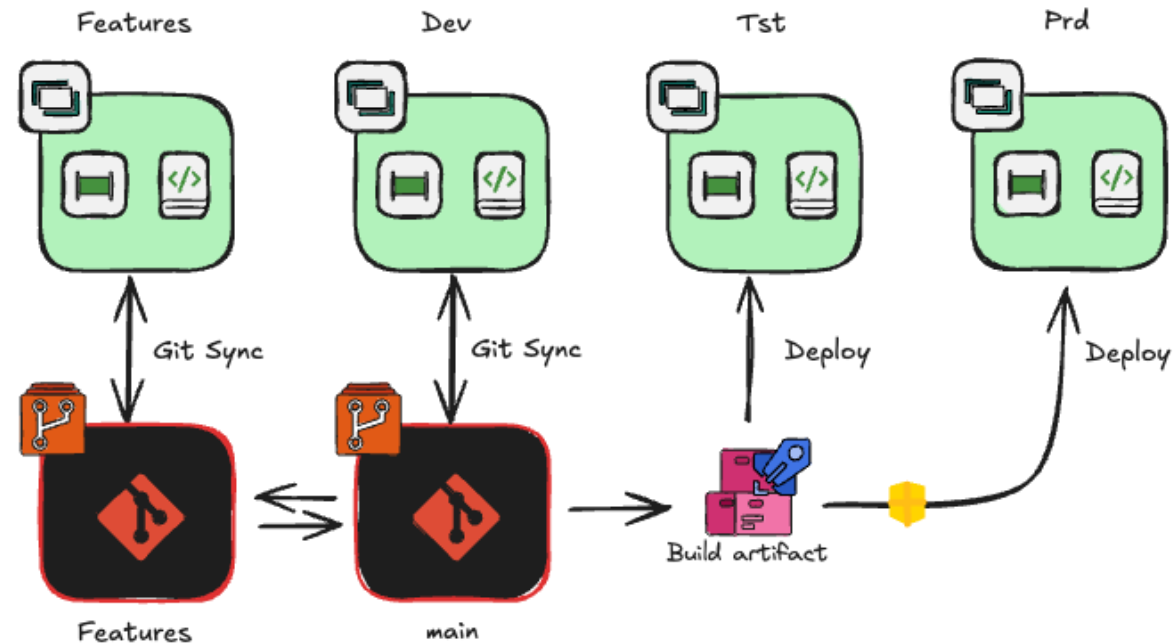
- Deploy all items (with public APIs)
- Auto-unpublish orphaned artifacts



RELEASE PROCESS - 3 SELECTED OPTIONS

Option 1

Dev = Main

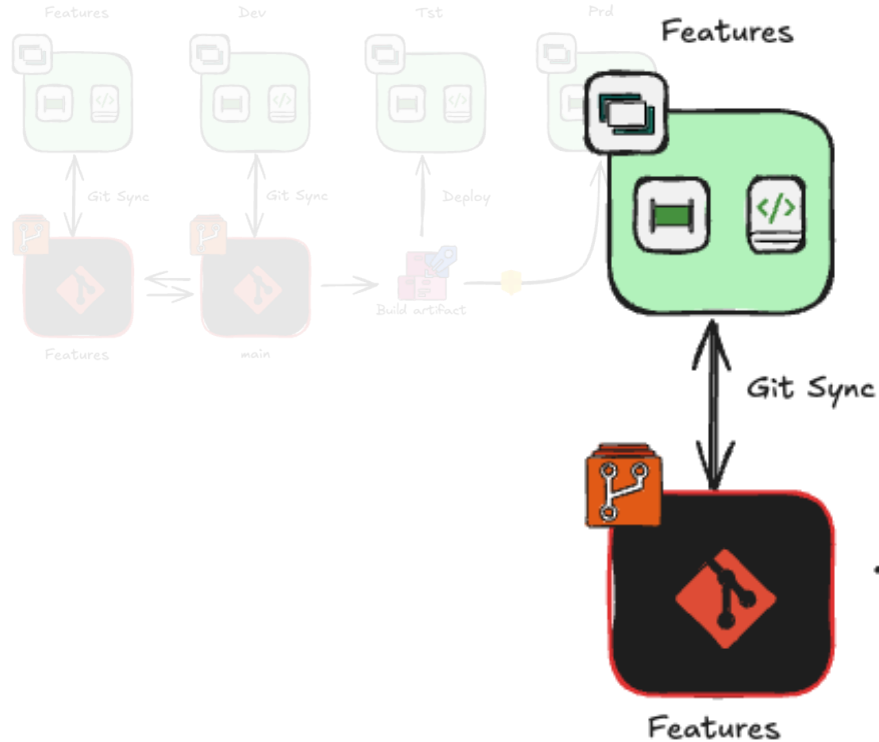




RELEASE PROCESS - 3 SELECTED OPTIONS

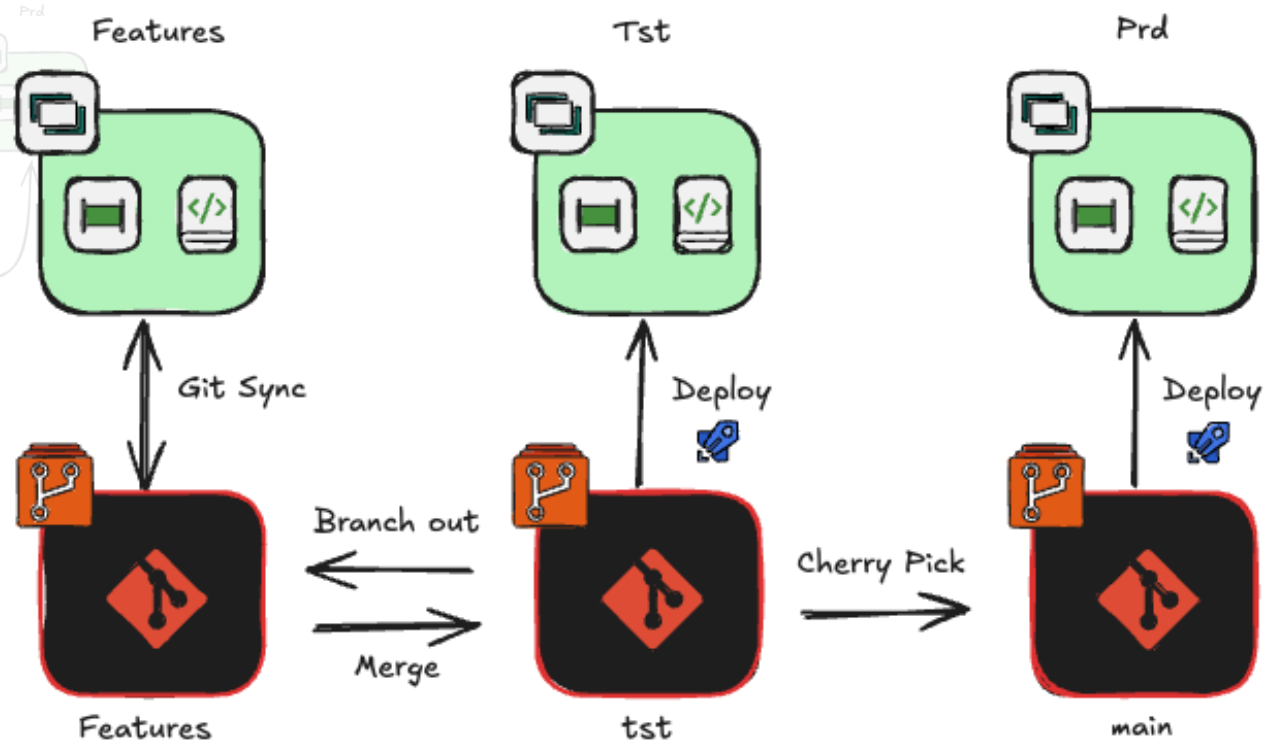
Option 1

Dev = Main



Option 2

PR to test – Cherry pick to main

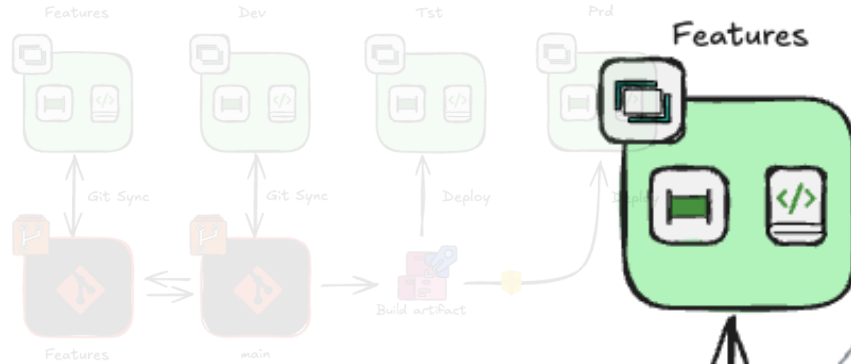




RELEASE PROCESS - 3 SELECTED OPTIONS

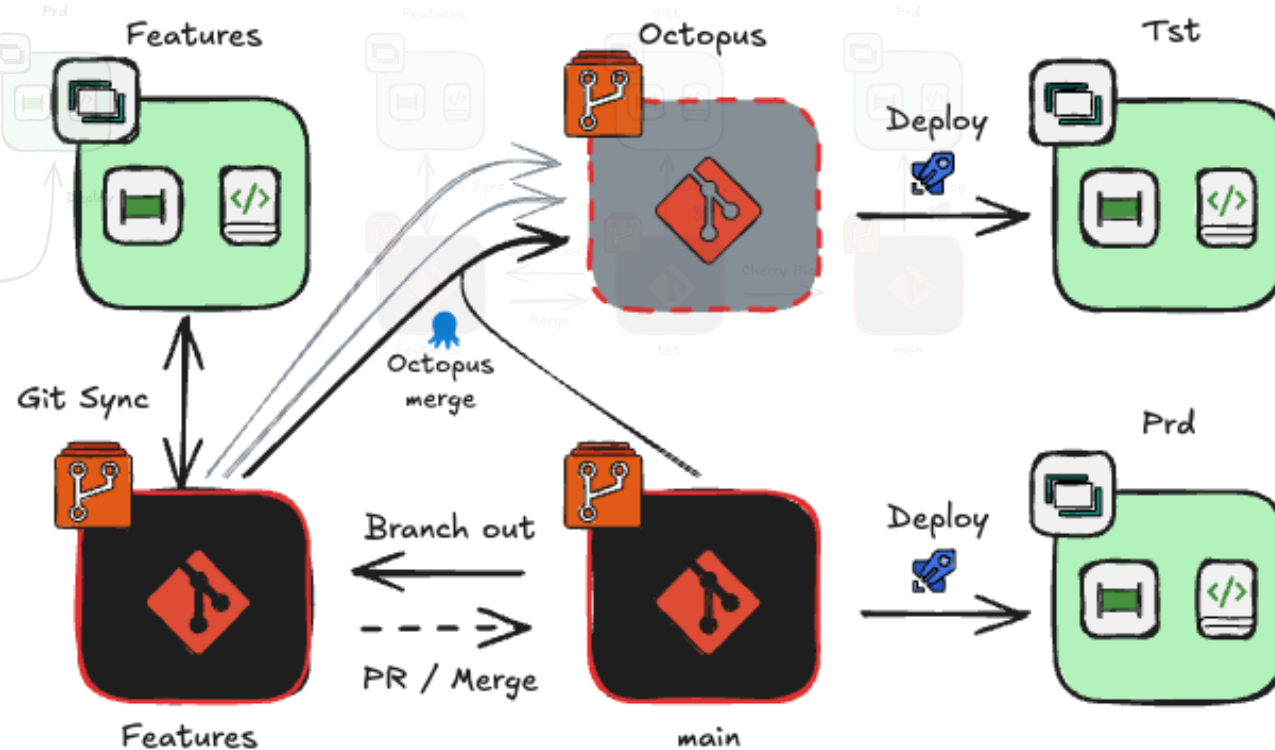
Option 1

Dev = Main



Option 3

PR to test Octopus deploy to main

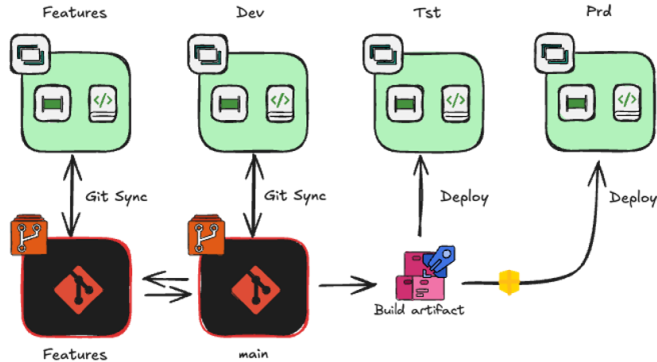




RELEASE PROCESS - 3 SELECTED OPTIONS

Option 1

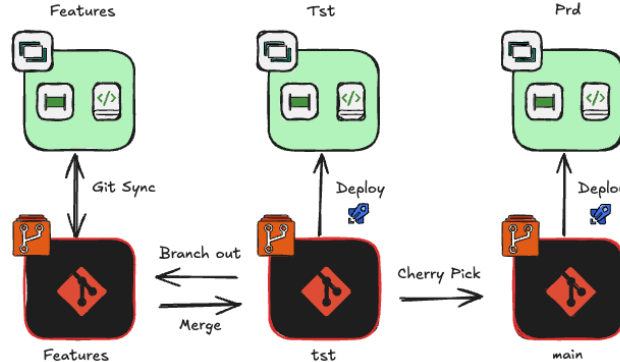
Dev = Main



- Smaller and more simple solutions
- Small teams
- No or very low interdependency between features

Option 2

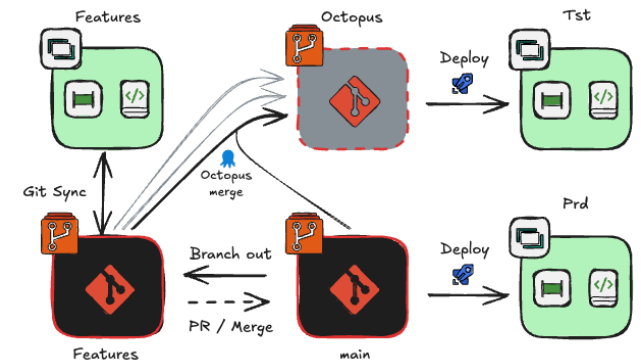
PR to test – Cherry pick to main



- Medium to large solutions
- High-risk changes
- Supports per-feature validation
- Supports incremental and selective testing and deployment

Option 3

Octopus deploy



- Large teams
- Many parallel features
- Interdependent changes
- High-risk changes
- Support validation and test



DEMO TIME!



TIPS, TRICKS & SUMMING UP





TIPS, TRICKS & SUMMING UP!

- Start using Git integration in Fabric if you are not doing it already
- Split your workloads and layers across multiple workspace
- Use mono-repo structure as a starting point
- Give thoughts to how to integrate workspaces with Git - Everything is not enterprise
- Leverage the Fabric CLI, REST APIs, and/or Terraform for automation
- Use the fabric-cicd Python library to deploy Fabric items
- Design everything with automation in mind
- Implement a metadata-driven framework for scalability robustness and speed
- Stay curious - Get inspired by what others build!





ACCESS THE FABRICOPS REPOSITORY



<https://github.com/gronnerup/FabricOps>



Disclaimer:

The solution demonstrated in this session is provided for **demonstration and educational purposes only**. It is **unsupported**, and there are no guarantees regarding functionality, stability, or future updates. You are free to use it **as-is** or modify it to fit your needs. Use at your own risk.

TIME FOR QUESTIONS!

BEFORE CONNECTION IS TERMINATED
BY PEER... OR ALEXANDER... OR LINDA...

